

Java I/O Streams: Serialization and Deserialization

Limitation of DataInputStream (DIS) and DataOutputStream (DOS)

Using `DataOutputStream` and `DataInputStream` classes we cannot read and write objects from persistence media. They only have the capability to read and write data up to primitive data types.

Solution

To read and write entire objects, we must use `ObjectInputStream` (OIS) and `ObjectOutputStream` (OOS).

ObjectInputStream and ObjectOutputStream

These two classes are used to store an object's state permanently in files or to send it to a remote computer via a network.

ObjectOutputStream class is a subclass of the `ObjectOutput` interface. It implements the below method to write an object to the underlying output stream:

```
public void writeObject(Object obj) throws IOException
```

ObjectInputStream class is a subclass of the `ObjectInput` interface. It implements the below method to read an object from the underlying input stream:

```
public Object readObject() throws IOException, ClassNotFoundException
```

Rule: To write or send an object to the external world, it must be of type `java.io.Serializable` interface. It means this object's class must implement the `java.io.Serializable` interface. Else, the `writeObject()` method throws a `java.io.NotSerializableException`.

Constructors to create OIS and OOS objects

Like DIS and DOS, OIS and OOS cannot connect to the source and destination directly. So their constructors take other input and output stream class objects as arguments.

```
public ObjectInputStream(InputStream in)
public ObjectOutputStream(OutputStream out)
```

Example: Serializing a Bank Object

```
public class Bank implements java.io.Serializable {

    static double minBalance = 5000;

    private long accNo;
    private String accHName;
    private String username;
    private transient String password;
    private transient double balance;

    public void setAccNo(long accNo) {
        this.accNo = accNo;
    }
    public long getAccNo() {
        return accNo;
    }

    public void setAccHName(String accHName) {
        this.acchName = accHName;
    }
    public String getAccHName() {
        return acchName;
    }

    public void setUsername(String username) {
        this.username = username;
    }
    public String getUsername() {
        return username;
    }

    public void setPassword(String password) {
        this.password = password;
    }
    public String getPassword() {
        return password;
    }

    public void setBalance(double amt) {
        this.balance = this.balance + amt;
    }
    public double getBalance() {
        return balance;
    }

    public String toString() {
        return "accNo: " + accNo + "\n" +
            "acchName: " + acchName + "\n" +
            "username: " + username + "\n" +
            "password: " + password + "\n" +
```

```

        "balance: " + balance + "\n" +
        "minBalance: " + minBalance + "\n";
    }
}

```

```

import java.io.*;

class BankTransaction {
    public static void main(String[] args) throws Exception {

        // 1. Creating bank object
        Bank acc1 = new Bank();

        // 2. Setting bank object state
        acc1.setAccNo(1);
        acc1.setAccHName("Hari");
        acc1.setUsername("Hari Krishna");
        acc1.setPassword("Naresh technologies");
        acc1.setBalance(999999999);

        // 3. Printing bank object state
        System.out.println(acc1);

        // 4. Creating OOS object wrapped around FOS
        ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("BankAccountsInfo.ser")
        );

        // 5. Writing bank object state to file
        oos.writeObject(acc1);

        System.out.println("Object written to file successfully.");
        oos.close();
    }
}

```

```
// BankUser.java -> shows object reading (Deserialization)
import java.io.*;

class BankUser {
    public static void main(String[] args) throws Exception {

        // 1. Creating OIS object wrapped around FIS
        ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("BankAccountsInfo.ser")
        );

        // 2. Casting returned Object to Bank type
        Bank accDetails = (Bank) ois.readObject();

        // 3. Printing Bank object data
        System.out.println(accDetails.getAccHName() + " details:");
        System.out.println(accDetails);

        ois.close();
    }
}
```

Notice that when you run `BankUser`, the `password` and `balance` will be printed as `null` and `0.0` because they were declared with the `transient` keyword!

What is the actual functionality of OOS and OIS?

Performing **Serialization** and **Deserialization**.

Serialization

Serialization is the process of converting objects into a stream of bytes and sending them to an underlying `OutputStream`. Using Serialization, we can store object state permanently in a destination, for example, a file or a remote computer.

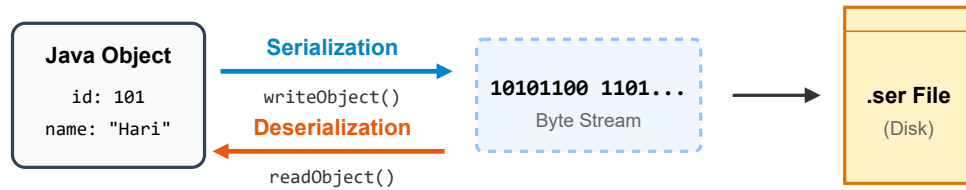
The serialization operation is performed by the `writeObject()` method of `ObjectOutputStream`.

Deserialization

Deserialization is the process of converting a stream of bytes back into the original object.

The deserialization operation is performed by the `readObject()` method of `ObjectInputStream`.

Rule on Serialization: Only `java.io.Serializable` type objects are serialized. `Serializable` is a marker interface; it doesn't have any methods. It provides special permission or identity to the JVM to serialize the object. If the given object isn't of type `Serializable` interface, then `writeObject()` will throw an unchecked exception called `java.io.NotSerializableException`.



Under the Hood: writeObject() and readObject()

Q) What is the functionality of writeObject() and readObject() methods?

In Serialization, the `writeObject()` method stores the object's state in the file with the following information:

- Its class name
- Current `.class` file's `serialVersionUID`
- Its non-transient variable names and their data types
- Those variables' currently modified values.

In Deserialization, the `readObject()` method creates the object with the state available in the file as shown below:

1. `readObject()` first reads the class name from the serialized file and loads that class by using `Class.forName()`. While loading the class, static variables get memory with originally assigned values and Static Blocks (SBs) are executed.
2. Then it compares the `serialVersionUID` of the currently loaded class with the `serialVersionUID` that is stored in the serialized file.
3. If both are **not the same**, it terminates the deserialization process by throwing `java.io.InvalidClassException`.
4. If both values **are the same**, it creates an object with the currently loaded class for non-static variables **without using the `new` keyword and constructor**.
5. Then the `readObject()` method populates values from the serialized file for the non-transient variables. It assigns default values for transient variables and also for any non-static variables that are not available in the serialized file.

Rules in Deserialization:

1. The object class's `.class` file must be available in the classpath; else `readObject()` throws `java.lang.ClassNotFoundException`.
2. We must use the exact same version of the `.class` file that was used in serialization (matching `serialVersionUID`), else the object is not deserialized.

Serialization with IS-A and HAS-A Relations

1. Serialization with IS-A relation (Inheritance)

If a superclass implements the `Serializable` interface, then all its subclasses are also automatically serializable types. They do not need to explicitly implement `Serializable` again.

In this case, if we serialize a subclass object, all the superclass's non-transient instance variables are also serialized. In deserialization, all subclass and superclass instance variable values are populated directly from the serialized file.

```
// A.java
class A implements java.io.Serializable {
    int x;
    A() {
        x = 50;
        System.out.println("x is initialized with 50");
    }
}

// B.java
class B extends A {
    int y;
    B() {
        y = 60;
        System.out.println("y is initialized with 60");
    }
    public String toString() {
        return "x: " + x + "\ny: " + y;
    }
}
```

```
// OOSDemo.java
import java.io.*;

public class OOSDemo {
    public static void main(String[] args) throws Exception {
        ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("test.ser"));

        B b = new B();
        b.x = 70;
        b.y = 80;

        System.out.println("x, y values are changed to 70, 80");
        oos.writeObject(b);
        System.out.println("B object is written to file");
    }
}
```

Q) If we remove `implements java.io.Serializable` from class A, will object B be written to file?

A) No, it leads to the exception `java.io.NotSerializableException`.

Q) If we remove `implements java.io.Serializable` from class A, and we add it ONLY to class B's declaration, will object B be written to file?

A) Yes, but **only** class B's non-transient variables' values are stored in the serialized file.
In deserialization, superclass A's instance variable 'x' is initialized by running A's no-arg constructor!

FAQ: Must a superclass be Serializable to serialize a subclass object?

A: No. But if the superclass is not serializable, it MUST have a no-arg constructor so the JVM can initialize the inherited variables during deserialization.

2. Serialization with HAS-A relationship (Composition)

Q) To serialize an object, must its internal composite objects also be Serializable?

A) Yes, otherwise the parent object will not be serialized and will throw a `NotSerializableException`.

```
class Student implements java.io.Serializable {
    String name = "Hari Krishna";

    // Address class is NOT Serializable!
    Address add = new Address();
}

class Address {
    String city = "Hyd";
}
```

Solutions to the HAS-A Serialization Problem:

1. **Declare `add` as `transient` :**

In this case, the deserialized `Student` object's `add` variable will have the value `null`. The student will lose their address. (Not a great solution).

2. **Make `Address` implement `Serializable` :**

This is the best approach if you have access to modify the `Address` source code.

3. **Customize Serialization:**

If you cannot modify the `Address` class, you can customize how `Student` serializes itself using private `writeObject` and `readObject` methods.

Customizing Serialization

If we want to write a `Student` object along with its current `Address` object values (even if `Address` is not serializable), we must customize the serialization by providing two very specific private methods inside the `Student` class.

```

class Student implements java.io.Serializable {
    int rollNum;
    String sname;
    transient Address add; // Must be transient so JVM doesn't try to serialize it normally

    Student(int rollNum, String sname, Address add) {
        this.rollNum = rollNum;
        this.sname = sname;
        this.add = add;
    }

    private void writeObject(ObjectOutputStream oos) {
        try {
            // 1. Perform default serialization for all non-transient variables
            oos.defaultWriteObject();

            // 2. Manually write the current Student's Address data
            oos.writeInt(add.hno);
            oos.writeUTF(add.city);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void readObject(ObjectInputStream ois) {
        try {
            // 1. Perform default deserialization for all non-transient variables
            ois.defaultReadObject();

            // 2. Manually read the Address state back in the EXACT same order
            int hno = ois.readInt();
            String city = ois.readUTF();

            // 3. Reconstruct the Address object manually
            add = new Address(hno, city);
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }

    public String toString() {
        return "rollNum: " + rollNum + "\n" +
            "sname: " + sname + "\n" +
            "hno: " + add.hno + "\n" +
            "city: " + add.city + "\n";
    }
}

// Note: Address does NOT implement Serializable
class Address {

```

```

int hno;
String city;

Address(int hno, String city) {
    this.hno = hno;
    this.city = city;
}
}

```

Functionality of private writeObject and readObject methods:

- These two methods are automatically called by the JVM (they act like callbacks during the serialization process).
- `private writeObject` is called from the `oos.writeObject()` method, passing the current OOS object reference as an argument.
- `private readObject` is called from the `ois.readObject()` method, passing the current OIS object reference as an argument.
- Inside these methods, we **must** call `oos.defaultWriteObject()` and `ois.defaultReadObject()` first to handle normal serialization of regular fields. Otherwise, the object's default data will be lost.
- Next, we use `writeXxx()` and `readXxx()` (like `writeInt()`, `writeUTF()`) to manually save and restore the non-serializable object's data.

Marker Interfaces

Q) What is a marker interface?

A) An interface that is used to check, mark, or tag a given object to indicate it possesses a specific property or to allow a special operation to be performed on that object. Marker interfaces do not declare any methods; they are empty interfaces.

Q) What is the need for a marker interface?

A) It provides metadata to the JVM or to specific utility methods. For example, if a class implements `java.io.Serializable`, the JVM knows it has permission to serialize it. If a class implements `java.lang.Cloneable`, the JVM knows it is permitted to create a clone of it.

Inside utility methods, the developer checks if the passed object implements the interface using the `instanceof` operator. For example, deep inside the Java source code for `ObjectOutputStream`, you will conceptually find code similar to this:

```

public class ObjectOutputStream extends OutputStream {
    public void writeObject(Object obj) {
        // Checking the marker interface
        if (obj instanceof Serializable) {
            // ... proceed with writing bytes to stream ...
        } else {
            throw new NotSerializableException(obj.getClass().getName());
        }
    }
}

```

Q) Can we write custom marker interfaces?

A) Yes. You just write an empty interface. Then, inside your business logic methods, you check if the passed object is of that type using the `instanceof` operator before proceeding.

Q) Is a marker interface an empty interface?

A) Yes.

Q) Is every empty interface a marker interface?

A) It depends on how it is used. If it is used in an `instanceof` operator condition to trigger special logic, then it is a marker interface. Otherwise, it is just an empty interface.